

[LINUX/WINDOWS] Manual — Engenharia de Software Assistida por IA com Kiro

Kiro + GitHub + GitHub Issues + GitHub Projects + Pull Requests + Automação +
Documentação Assistida por IA

1. Objetivo da atividade

Nesta atividade, vamos utilizar IA para acelerar o fluxo de engenharia de software utilizando:

- Kiro
- GitHub
- GitHub Issues
- GitHub Projects
- Pull Requests
- GitHub CLI
- Automação via scripts
- Engenharia orientada por backlog

O objetivo é transformar um texto não estruturado, como uma ata de reunião, em um fluxo completo de desenvolvimento:

Ata de reunião
→ backlog estruturado
→ Epic
→ Stories
→ checklist técnico
→ GitHub Issues
→ branches
→ commits padronizados
→ Pull Requests
→ merge
→ documentação automática

A IA acelera o processo, mas a validação continua sendo humana.

2. Ferramentas utilizadas

- Git
- GitHub
- GitHub CLI ([gh](#))
- Visual Studio Code
- Kiro
- GitHub Issues
- GitHub Projects
- Pull Requests

Modelos recomendados no Kiro:

- Auto
 - Claude Sonnet
 - Qwen3 Coder
 - DeepSeek
-

3. Instalação no Linux

3.1 Instalar Git

```
sudo apt update  
sudo apt install git -y
```

Testar:

```
git --version
```

3.2 Instalar VSCode

Baixe:

[Visual Studio Code](#)

Depois teste:

```
code --version
```

3.3 Instalar GitHub CLI

```
sudo apt install curl ca-certificates -y

sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.gpg | \
sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null

sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg] https://cli.github.com/packages
stable main" | \
sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null

sudo apt update
sudo apt install gh -y
```

Testar:

```
git --version
```

3.4 Login no GitHub CLI

```
gh auth login
```

Escolha:

```
GitHub.com
HTTPS
Login with a web browser
```

Depois valide:

```
gh auth status
```

3.5 Instalar Kiro CLI

Instalar:

```
curl -fsSL https://cli.kiro.dev/install | bash
```

Adicionar ao PATH:

```
export PATH="$HOME/.local/bin:$PATH"
```

Persistir:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc  
source ~/.bashrc
```

Validar:

```
which kiro-cli
```

3.6 Login no Kiro

```
kiro-cli login
```

Depois valide:

```
kiro-cli whoami
```

4. Instalação no Windows

4.1 Instalar Git

Baixe:

[Git for Windows](#)

Depois teste:

```
git --version
```

4.2 Instalar VSCode

Baixe:

[Visual Studio Code](#)

Durante a instalação marque:

- Add to PATH
 - Open with Code
-

4.3 Instalar GitHub CLI

Baixe:

[GitHub CLI](#)

Depois teste:

```
gh --version
```

4.4 Login no GitHub CLI

```
gh auth login
```

Escolha:

```
GitHub.com
HTTPS
Login with a web browser
```

Depois valide:

```
gh auth status
```

4.5 Instalar Kiro CLI

Baixe:

[Kiro CLI](#)

Depois valide:

```
kiro-cli whoami
```

4.6 Ambiente recomendado no Windows

Utilize:

- Git Bash
- PowerShell

Evite:

- CMD puro
-

5. Instalar Kiro no VSCode

No VSCode:

```
Ctrl + Shift + X
```

Pesquisar:

```
Kiro
```

Instalar:

```
Kiro — AI-Powered Development For VS Code
```

Depois reinicie o VSCode.

6. Configuração recomendada do Kiro

No canto superior do chat do Kiro:

Situação	Modelo
----------	--------

Uso geral	Auto
Engenharia de software	Claude Sonnet
Refatoração pesada	Claude Opus
Código rápido	DeepSeek
Scripts	Qwen3 Coder

7. Criar repositório

Criar repositório:

```
ai-software-engineering-demo
```

Marcar:

```
Add README
```

8. Clonar repositório

```
git clone https://github.com/SEU\_USUARIO/ai-software-engineering-demo.git  
cd ai-software-engineering-demo
```

Abrir VSCode:

```
code .
```

9. Estrutura do projeto

Estrutura esperada:

```
ai-software-engineering-demo/  
|  
├── .github/
```

```
|
| |--- ISSUE_TEMPLATE/
| | |--- user_story.yml
| |--- instructions/
| | |--- docs.instructions.md
| |--- ai-instructions.md
| |--- pull_request_template.md
|
|--- docs/
| |--- templates/
| | |--- README_TEMPLATE.md
| | |--- CONTRIBUTING_TEMPLATE.md
| |--- meeting-notes.md
| |--- product.md
| |--- backlog.md
| |--- PRD.md
| |--- prompts.md
|
|--- scripts/
| |--- create_issues.sh
| |--- create_docs_issue.sh
| |--- start_issue.sh
| |--- open_pr.sh
|
|--- src/
| |--- search.js
|
|--- CONTRIBUTING.md
|--- README.md
```

10. Criar estrutura de pastas

Linux

```
mkdir -p .github/ISSUE_TEMPLATE
mkdir -p .github/instructions
mkdir -p docs/templates
mkdir -p docs
mkdir -p scripts
mkdir -p src

touch .github/ISSUE_TEMPLATE/user_story.yml
touch .github/pull_request_template.md
```



```
touch .github/ai-instructions.md
touch .github/instructions/docs.instructions.md

touch docs/meeting-notes.md
touch docs/product.md
touch docs/backlog.md
touch docs/templates/README_TEMPLATE.md
touch docs/templates/CONTRIBUTING_TEMPLATE.md

touch scripts/start_issue.sh
touch scripts/open_pr.sh

touch CONTRIBUTING.md
touch README.md
touch src/search.js
```

Windows PowerShell

```
mkdir .github\ISSUE_TEMPLATE -Force
mkdir .github\instructions -Force
mkdir docs\templates -Force
mkdir scripts -Force
mkdir src -Force

ni .github\ISSUE_TEMPLATE\user_story.yml -ItemType File
ni .github\pull_request_template.md -ItemType File
ni .github\ai-instructions.md -ItemType File
ni .github\instructions\docs.instructions.md -ItemType File

ni docs\meeting-notes.md -ItemType File
ni docs\product.md -ItemType File
ni docs\backlog.md -ItemType File
ni docs\templates\README_TEMPLATE.md -ItemType File
ni docs\templates\CONTRIBUTING_TEMPLATE.md -ItemType File

ni scripts\start_issue.sh -ItemType File
ni scripts\open_pr.sh -ItemType File

ni CONTRIBUTING.md -ItemType File
ni README.md -ItemType File
ni src\search.js -ItemType File
```

11. Arquivo de ata da reunião

Arquivo:

docs/meeting-notes.md

Conteúdo:

Ata de reunião — Shop4u

O sistema Shop4u será um aplicativo mobile de e-commerce.

Funcionalidades discutidas:

- Busca de produtos por nome e categoria
- Carrinho de compras
- Checkout com autenticação
- Recomendações personalizadas com IA
- Notificações de pedido

Regras de negócio:

- Usuário deve estar autenticado para finalizar compra
- Recomendações devem considerar histórico de navegação
- Usuários sem histórico devem receber produtos populares
- Pedido só deve ser criado após pagamento aprovado
- Se o pagamento for recusado, o carrinho deve permanecer ativo

O time deseja utilizar IA para acelerar:

- criação de backlog
- documentação
- Pull Requests
- convenções de desenvolvimento

12. Arquivo de produto

Arquivo:

docs/product.md

Conteúdo:

Shop4u

Aplicativo mobile de e-commerce com recomendação por IA.

Objetivo

Criar uma experiência de compra simples, personalizada e eficiente.

Funcionalidades principais

- Buscar produtos
- Receber recomendações personalizadas
- Gerenciar carrinho
- Finalizar compra
- Receber confirmação do pedido

13. README inicial mínimo

Arquivo:

README.md

Conteúdo:

ai-software-engineering-demo

Projeto demonstrativo de engenharia de software assistida por IA.
O README será expandido posteriormente com apoio do Kiro.

14. CONTRIBUTING inicial mínimo

Arquivo:

CONTRIBUTING.md

Conteúdo:

Guia de contribuição

Este arquivo será atualizado automaticamente com apoio do Kiro.

15. Template de User Story

Arquivo:

.github/ISSUE_TEMPLATE/user_story.yml

Conteúdo:

```
name: User Story
description: Template para criação de User Stories
title: "[STORY] "
labels: ["story"]

body:
- type: textarea
  id: user-story
  attributes:
    label: User Story
    description: Descreva no formato Como / Quero / Para.
    placeholder: |
      Como cliente,
      quero buscar produtos,
      para encontrar itens rapidamente.
  validations:
    required: true

- type: textarea
  id: bdd
  attributes:
    label: Critérios de aceitação — BDD
    description: Use Dado / Quando / Então.
    placeholder: |
      Cenário 1:

      Dado que ...
      Quando ...
      Então ...
  validations:
    required: true
```

```
- type: textarea
id: checklist
attributes:
  label: Checklist técnico
  description: Liste tarefas técnicas.
  placeholder: |
    - [ ] Criar endpoint
    - [ ] Criar frontend
```

16. Template de Pull Request

Arquivo:

```
.github/pull_request_template.md
```

Conteúdo:

O que foi feito

Descreva brevemente a alteração realizada.

Issue relacionada

Closes #

Validação da Story

- [] Todos os itens aplicáveis do checklist técnico da Issue foram concluídos
- [] Os critérios BDD foram considerados
- [] A Issue relacionada está pronta para ser fechada

Checklist

- [] A User Story foi considerada
- [] Os critérios BDD foram considerados
- [] O código foi testado
- [] A documentação foi atualizada, se necessário

17. Instruções permanentes para IA

Arquivo:

.github/ai-instructions.md

Conteúdo:

Instruções do projeto para IA

Este repositório é uma demonstração de Engenharia de Software Assistida por IA usando GitHub, GitHub Issues, GitHub Projects, branches, Pull Requests e documentação automatizada.

Objetivo do projeto

O projeto simula o desenvolvimento do Shop4u, um aplicativo mobile de e-commerce com recomendação por IA.

Regras gerais

- Use linguagem técnica, objetiva e didática.
- Não invente funcionalidades que não existam no repositório.
- Ao gerar documentação, considere apenas arquivos existentes e comportamentos implementados.
- Ao gerar Issues, use Markdown com quebras reais.
- Nunca use \n escapado no corpo de Issues.
- Para Issues via terminal, use --body-file ou heredoc.
- Não execute comandos automaticamente sem autorização explícita.

Backlog

Use a seguinte estrutura:

- [EPIC]
- [STORY]
- [DOCS]

Tasks pequenas devem ficar como checklist técnico dentro da Story.

Fluxo de desenvolvimento

1. Escolher uma Issue
2. Rodar ./scripts/start_issue.sh
3. Implementar alteração

4. Fazer commit
5. Rodar ./scripts/open_pr.sh
6. Revisar Pull Request
7. Fazer merge
8. Apagar branch

18. Instruções específicas para documentação

Arquivo:

```
.github/instructions/docs.instructions.md
```

Conteúdo:

```
---
applyTo: "**/*.md"
---

# Instruções para documentação

## README.md

O README deve conter:

- Nome do projeto
- Objetivo
- Visão geral
- Funcionalidades
- Estrutura de pastas
- Como executar
- Fluxo de desenvolvimento
- Convenção de commits

## CONTRIBUTING.md

O CONTRIBUTING deve conter:

- Fluxo de contribuição
- Como escolher Issues
- Como abrir branches
```

- Como abrir Pull Requests
- Como revisar checklist
- Boas práticas

Estilo

- Linguagem técnica
- Markdown organizado
- Não inventar funcionalidades

19. Template README

Arquivo:

docs/templates/README_TEMPLATE.md

Conteúdo:

Nome do projeto

Objetivo

Descreva o objetivo do projeto.

Visão geral

Descreva o contexto do sistema e o problema que ele resolve.

Funcionalidades

- Funcionalidade 1
- Funcionalidade 2
- Funcionalidade 3

Estrutura de pastas

```text

```
.
├── .github/
├── docs/
├── scripts/
├── src/
└── CONTRIBUTING.md
```



```
└─ README.md
...

Como executar localmente

```bash
# adicionar comandos necessários
```

Fluxo de desenvolvimento

```text
Issue
→ branch
→ commit
→ Pull Request
→ merge
```

Scripts disponíveis

Iniciar trabalho em uma Issue

```bash
./scripts/start_issue.sh
```

Abrir Pull Request

```bash
./scripts/open_pr.sh
```

Convenção de commits

```text
feat: nova funcionalidade
fix: correção
docs: documentação
chore: configuração
```

Documentação adicional
```

## 20. Template do CONTRIBUTING

**Arquivo:**

```
docs/templates/CONTRIBUTING_TEMPLATE.md
```

**Conteúdo:**

```
Guia de contribuição
```

```
Fluxo de contribuição
```

1. Escolha uma Issue aberta.
2. Inicie uma branch usando `scripts/start\_issue.sh`.
3. Implemente a alteração.
4. Faça commit seguindo a convenção.
5. Abra um Pull Request usando `scripts/open\_pr.sh`.
6. Revise o checklist.
7. Faça merge após validação.
8. Apague a branch após o merge.

```
Como iniciar uma Issue
```

```
```bash
./scripts/start_issue.sh
```
```

```
Como fazer commit
```

```
```bash
git add .
git commit -m "feat: descricao da alteracao"
```
```

```
Como abrir Pull Request
```

```
```bash
./scripts/open_pr.sh
```
```

```
Padrão de branches
```

```
```text
feature/<issue-id>-descricao
docs/<issue-id>-descricao
fix/<issue-id>-descricao
```

```
...
```

```
## Padrão de commits
```

```
``text
```

```
feat: nova funcionalidade
```

```
fix: correção
```

```
docs: documentação
```

```
chore: configuração
```

```
test: testes
```

```
refactor: refatoração
```

```
...
```

```
## Checklist antes do merge
```

- [] A Issue relacionada está correta.
- [] O PR contém `Closes #ID`.
- [] A User Story foi considerada.
- [] Os critérios BDD foram atendidos.
- [] O código foi testado.
- [] A documentação foi atualizada, quando necessário.

21. Criar labels no GitHub

Coloque cada comando um por um no terminal:

```
gh label create epic --description "Agrupador de alto nível" --color 5319E7 || true
gh label create story --description "Unidade principal de entrega" --color 0E8A16 || true
gh label create docs --description "Documentação" --color 1D76DB || true
gh label create backend --description "Backend" --color 0052CC || true
gh label create frontend --description "Frontend" --color FBCA04 || true
gh label create ai --description "Funcionalidades de IA" --color 8A2BE2 || true
gh label create "priority:high" --description "Alta prioridade" --color B60205 || true
gh label create "priority:medium" --description "Média prioridade" --color D93F0B || true
gh label create "priority:low" --description "Baixa prioridade" --color 0E8A16 || true
```

As labels `priority:high`, `priority:medium` e `priority:low` devem ficar entre aspas porque contêm `:`.

22. Commit inicial

```
git add .
git commit -m "chore: configura estrutura inicial"
git push
```

23. Criar backlog automaticamente com Kiro

Abrir Kiro Chat.

Modelo recomendado:

Claude Sonnet

Prompt:

Leia docs/meeting-notes.md e gere um backlog inicial para GitHub Issues do projeto Shop4u.

Gere exatamente:

- 1 Epic
- 4 Stories
- 1 Docs

Regras:

- A Epic representa o objetivo macro.
- As Stories representam entregas funcionais.
- A issue Docs representa documentação.
- Não crie tarefas técnicas como Issues separadas.
- Tarefas técnicas devem ficar como checklist dentro das Stories.
- Critérios de aceitação devem usar BDD com Dado / Quando / Então.

Títulos:

- [EPIC] ...
- [STORY] ...
- [DOCS] ...

Labels permitidas:

epic, story, docs, backend, frontend, ai, priority:high, priority:medium, priority:low

Cada Story deve conter:

- Parent Epic: #ID_DA_EPIC
- ## User Story

- ## Critérios de aceitação — BDD
- ## Checklist técnico

Regras obrigatórias para o script:

- Gere scripts/create_issues.sh completo.
- Crie primeiro a Epic.
- Capture a URL retornada pelo gh issue create.
- Extraia o número da Epic com sed.
- Use esse número como Parent Epic nas Stories.
- Use mktemp + --body-file para preservar Markdown.
- Não use \n escapado.
- Não use --body \$"texto\ntexto".
- Não use --json nem --jq com gh issue create.
- Não execute o script automaticamente.

Entrega:

1. Mostre primeiro as 6 Issues sugeridas para revisão.
2. Depois crie o arquivo scripts/create_issues.sh completo.

24. Executar criação das Issues

Depois que o Copilot gerar `scripts/create_issues.sh`, revise o arquivo.

Execute:

```
chmod +x scripts/create_issues.sh
./scripts/create_issues.sh
```

Depois valide:

```
gh issue list --limit 20
```

25. Criar GitHub Project

No GitHub:

```
Projects → New Project → Board
```

Colunas:

Backlog
Ready
In Progress
Review
Done

Adicione as Issues ao Project.

26. Script start_issue.sh

Arquivo:

scripts/start_issue.sh

Conteúdo:

```
#!/bin/bash

set -e

echo "Issues abertas:"
echo

gh issue list \
  --state open \
  --json number,title,labels \
  --jq '.[] | "\(.number) - \(.title)'"

echo
read -p "Digite o número da Issue: " ISSUE_ID

TITLE=$(gh issue view "$ISSUE_ID" --json title --jq '.title')

SLUG=$(echo "$TITLE" \
  | sed -E 's/\[(STORY|EPIC|DOCS|TECH|BUG)\]//g' \
  | tr '[:upper:]' '[:lower:]' \
  | iconv -f utf8 -t ascii//TRANSLIT \
  | sed -E 's/[^a-z0-9]+/-/g' \
  | sed -E 's/^-|-$/g')

TYPE="feature"
```

```
if [[ "$TITLE" == *"[DOCS]"* ]]; then
    TYPE="docs"
elif [[ "$TITLE" == *"[BUG]"* ]]; then
    TYPE="fix"
fi

BRANCH="${TYPE}/${ISSUE_ID}-${SLUG}"

echo
echo "Issue selecionada:"
echo "#$ISSUE_ID - $TITLE"
echo
echo "Branch:"
echo "$BRANCH"

git checkout main
git pull
git checkout -b "$BRANCH"
```

Esse script lista todas as Issues abertas, não apenas Stories. Assim ele também permite trabalhar em Issues [\[DOCS\]](#).

27. Script open_pr.sh

Arquivo:

```
scripts/open_pr.sh
```

Conteúdo:

```
#!/bin/bash

set -e

BRANCH=$(git branch --show-current)

ISSUE_ID=$(echo "$BRANCH" | sed -E 's#^[a-zA-Z]+/([0-9]+)-.*#\1#')

if [ -z "$ISSUE_ID" ] || [ "$ISSUE_ID" = "$BRANCH" ]; then
    echo "Erro: não foi possível identificar o número da Issue pela branch."
    echo "Use o padrão: feature/6-busca-produtos, docs/6-atualizar-readme ou fix/6-corrigir-bug"
    exit 1
fi
```

```

fi

ISSUE_TITLE=$(gh issue view "$ISSUE_ID" --json title --jq '.title')

PR_TITLE=$(echo "$ISSUE_TITLE" \
| sed -E 's/\[(STORY|EPIC|DOCS|TECH|BUG)\]//g' \
| sed -E 's/^ *//g')

COMMIT_TYPE="feat"

if [[ "$BRANCH" == docs/* ]]; then
    COMMIT_TYPE="docs"
elif [[ "$BRANCH" == fix/* ]]; then
    COMMIT_TYPE="fix"
fi

BODY_FILE=$(mktemp)

cat > "$BODY_FILE" <<EOF
## O que foi feito

Implementação relacionada à Issue #$ISSUE_ID.

## Issue relacionada

Closes #$ISSUE_ID

## Validação da Issue

- [ ] Todos os itens aplicáveis do checklist técnico da Issue foram concluídos
- [ ] Os critérios BDD foram considerados, quando aplicável
- [ ] A Issue relacionada está pronta para ser fechada

## Checklist

- [ ] A User Story foi considerada, quando aplicável
- [ ] O código foi testado, quando aplicável
- [ ] A documentação foi atualizada, quando necessário
EOF

git push -u origin "$BRANCH"

gh pr create \
--base main \
--head "$BRANCH" \
--title "$COMMIT_TYPE: $PR_TITLE" \
--body-file "$BODY_FILE"

```



```
rm "$BODY_FILE"
```

28. Tornar scripts executáveis

```
chmod +x scripts/start_issue.sh  
chmod +x scripts/open_pr.sh
```

29. Subir scripts

```
git add .  
git commit -m "chore: adiciona scripts de automacao"  
git push
```

30. Fluxo de desenvolvimento

Iniciar Issue

```
./scripts/start_issue.sh
```

O script lista as Issues abertas. Escolha o número da Issue.

Exemplo:

```
2 - [STORY] Busca e navegação de produtos  
6 - [DOCS] Documentação inicial
```

Depois de escolher, o script cria a branch automaticamente.

Implementar código

Exemplo:

Arquivo:

```
src/search.js
```

Conteúdo:

```
function searchProducts(products, query) {  
  return products.filter(product =>  
    product.name.toLowerCase().includes(query.toLowerCase())  
  );  
}  
  
module.exports = searchProducts;
```

Commit

```
git add .  
git commit -m "feat: implementa busca de produtos"
```

Abrir Pull Request automaticamente

```
./scripts/open_pr.sh
```

Esse script:

- faz push
- cria PR
- adiciona **Closes #ID** automaticamente

31. Merge

No GitHub:

```
Merge pull request
```

Após merge:

- código entra na **main**
- Issue fecha automaticamente

Atenção: o GitHub fecha a Issue quando encontra **Closes #ID** no PR mergeado. Ele não verifica automaticamente se o checklist técnico da Issue foi marcado. Por isso, a revisão humana antes do merge continua sendo importante.

32. Apagar branch

No GitHub, clique em:

Delete branch

Ou pelo terminal:

```
git push origin --delete feature/2-busca-e-navegacao-de-produtos-mobile-api  
git branch -d feature/2-busca-e-navegacao-de-produtos-mobile-api
```

33. Documentação automática com Kiro

Prompt:

Atualize README.md e CONTRIBUTING.md usando:

- .github/ai-instructions.md
- docs/templates/
- scripts/
- src/
- docs/

Não invente funcionalidades inexistentes.

Não faça:

- branch
- commit
- push
- PR

Antes de alterar os arquivos, mostre resumo das mudanças.

34. Criar novas Issues de documentação

Além da Issue [\[DOCS\]](#) inicial, o projeto pode possuir Issues específicas para documentar módulos implementados.

Exemplos:

```
[DOCS] Documentar módulo de busca
[DOCS] Documentar recomendações
[DOCS] Documentar fluxo de checkout
```

Para isso, crie o arquivo:

```
scripts/create_docs_issue.sh
```

Conteúdo:

```
#!/bin/bash

echo
read -p "Título da documentação: " DOC_TITLE

BODY_FILE=$(mktemp)

cat > "$BODY_FILE" <<EOF
## Objetivo

Documentar tecnicamente o módulo.

## Checklist técnico

- [ ] Criar ou atualizar documentação em docs/
- [ ] Revisar aderência com o código real
- [ ] Validar exemplos de uso
- [ ] Revisar organização do Markdown
EOF

gh issue create \
--title "[DOCS] ${DOC_TITLE}" \
--body-file "$BODY_FILE" \
--label docs

rm "$BODY_FILE"
```

Tornar executável:

```
chmod +x scripts/create_docs_issue.sh
```

Executar:

```
./scripts/create_docs_issue.sh
```

Exemplo:

Título da documentação:

Documentar módulo de busca

Arquivo/módulo relacionado:

src/search.js

Depois:

```
./scripts/start_issue.sh
```

Escolha a nova Issue criada.

35. Prompt para documentar código

Depois de iniciar a Issue [\[DOCS\]](#), utilize o Copilot Chat.

Prompt recomendado:

Documente o código em src/.

Considere:

- .github/ai-instructions.md
- docs/templates/
- src/

Use linguagem técnica.

Não invente funcionalidades.

Não:

- criar branch
- commit
- push
- PR

Exemplo específico (search-module.md):

Analisar o arquivo `src/search.js` e gerar documentação técnica.

Incluir:

- objetivo da função
- parâmetros
- retorno
- exemplo de uso
- limitações

Se fizer sentido, criar ou atualizar `docs/search-module.md`.

Importante:

- Não criar branch.
- Não fazer commit.
- Não fazer push.
- Não abrir Pull Request.
- Apenas gerar ou atualizar os arquivos de documentação.

Depois que o Copilot sugerir as alterações:

1. Revisar
2. Clicar em Manter

Depois:

```
git add .  
git commit -m "docs: documenta modulo de busca"  
./scripts/open_pr.sh
```

36. Geração automática do PRD

Objetivo do PRD

O PRD (Product Requirements Document) descreve:

- visão do produto
- problema resolvido
- funcionalidades
- regras de negócio
- fluxos
- requisitos
- arquitetura funcional

- critérios de sucesso

O arquivo deve ficar em:

docs/PRD.md

Fluxo recomendado

1. Criar Issue [DOCS]
2. Iniciar branch com:

./scripts/start_issue.sh
3. Usar o Copilot Chat para gerar o PRD
4. Revisar manualmente
5. Commitar
6. Abrir PR
7. Fazer merge

Prompt:

Crie docs/PRD.md para o projeto Shop4u.

Estrutura:

- visão geral
- problema
- objetivos
- funcionalidades
- regras de negócio
- requisitos
- arquitetura
- stack
- critérios de sucesso

Não invente funcionalidades inexistentes.

37. Registro de prompts

Arquivo:

docs/prompts.md

Objetivo:

Registrar:

- prompts usados no Copilot
- prompts usados para arquitetura
- prompts usados para testes
- prompts usados para documentação
- prompts usados para backlog

Prompt para gerar automaticamente:

Crie docs/prompts.md documentando os prompts usados neste projeto.

Organize:

- backlog
- UML
- documentação
- testes
- PRD

38. Estrutura recomendada da documentação

```
docs/
|
|— architecture.md
|— api.md
|— setup.md
|— development-flow.md
|— search-module.md
|— checkout-module.md
|— recommendation-module.md
```

A documentação deve acompanhar a evolução real do código.

39. Fluxo final completo

Ata de reunião

- backlog gerado por IA
- Epic e Stories
- checklist técnico

- GitHub Issues
- branch automática
- implementação
- documentação automática
- Pull Request automática
- merge
- fechamento automático da Issue

Fluxo completo para documentação técnica:

Issue [DOCS]

- ./scripts/create_docs_issue.sh
- ./scripts/start_issue.sh
- Copilot gera documentação
- revisão humana
- commit
- ./scripts/open_pr.sh
- merge
- fechamento automático da Issue

40. Conceitos importantes

Epic

Agrupa várias Stories relacionadas.

Exemplo:

[EPIC] Lançamento MVP Shop4u

Story

É a unidade principal de entrega.

Exemplo:

[STORY] Busca e navegação de produtos

Docs

Representa entregas de documentação.

Exemplo:

[DOCS] Documentar módulo de busca

Checklist técnico

Representa as tasks necessárias para concluir a Story.

Essas tasks ficam dentro da Story. Não precisam virar sub-issues, a menos que o trabalho seja grande ou precise ser distribuído entre múltiplos responsáveis.

Labels

As labels são atribuídas na criação da Issue. Elas indicam tipo, área e prioridade.

Exemplo:

story
backend
frontend
priority:high

Critérios sugeridos:

priority:high = fluxo crítico do MVP
priority:medium = importante, mas não bloqueia o MVP
priority:low = complementar ou melhoria futura

A prioridade sugerida pela IA deve ser revisada pelo time.

41. Responsabilidade humana

A IA não substitui engenharia.

Ela acelera:

backlog
documentação
organização
convenções
Pull Requests
rastreadabilidade

Mas continua sendo responsabilidade do desenvolvedor:

validar regras de negócio
revisar código
revisar documentação
garantir qualidade
aprovar alterações
verificar se a Issue realmente está pronta antes do merge